MIC: A High-Throughput Lossless Codec for Medical Images

Kuldeep Singh*

Abstract

We present MIC (Medical Image Codec), a lossless compression codec optimized for 16-bit medical images stored in the DICOM format. MIC chains three lightweight stages—spatial delta prediction, run-length encoding (RLE), and finite state entropy (FSE) coding based on asymmetric numeral systems (ANS)—to achieve compression ratios of 1.7×–8.6× on greyscale DICOM images and 3–5× on RGB whole slide imaging (WSI) tissue tiles (excluding degenerate constant-plane background tiles). The codec supports three container formats: MIC1 for single frames, MIC2 for multi-frame sequences with optional temporal prediction, and MIC3 for tiled whole slide images with multi-resolution pyramid levels. A four-state FSE decoder breaks the serial dependency chain inherent in ANS decoding by running four independent state machines, with platform-specific BMI2 (amd64) and scalar (arm64) assembly kernels, yielding 1.7–2.4× faster end-to-end single-threaded decompression. A fourth format, PICS (Parallel Image Compressed Strips), partitions a single image into N horizontal strips and compresses each strip independently, enabling multi-core decompression of a single greyscale image at 2.5–3.9× single-thread throughput on images ≥7 MB. We evaluate MIC on eight clinical DICOM datasets spanning MR, CT, CR, X-ray, and mammography modalities. Using fair in-process benchmarking via CGO bindings, we compare against HTJ2K (OpenJPH), JPEG-LS (CharLS), and Delta+Zstandard. HTJ2K (OpenJPH) decompresses 1.8× faster than MIC-Go on a single thread; however, MIC's C implementation with four-state FSE closes to within 0.87× of HTJ2K, and MIC's wavelet SIMD pipeline exceeds HTJ2K on all eight modalities. JPEG-LS achieves 1–16% better compression ratios due to its context-adaptive MED predictor, but MIC decompresses 1.4–2.6× faster (2-state) and 1.5–3.5× faster (4-state) on six of eight modalities. MIC's 16-bit-native RLE+FSE pipeline outperforms Delta+Zstandard by 10–22% on all modalities. A five-level 5/3 integer wavelet alternative matches or exceeds both Delta+RLE+FSE and HTJ2K compression ratios on seven of eight modalities; its SIMD-accelerated C implementation (blocked column pass + AVX2 lifting + four-state FSE) exceeds HTJ2K decompression speed on all eight modalities, though it doubles memory bandwidth relative to the delta pipeline. MIC's per-frame parallelism scales to 16 GB/s on mammography (best case; geometric mean ≈7.5 GB/s across modalities on 64 cores). MIC is implemented in Go with JavaScript and WebAssembly browser decoders, and is released as open-source software.

1 Introduction

Medical imaging generates enormous volumes of data. A single digital breast tomosynthesis (DBT) study produces 69 or more frames of 2457×1890 pixels at 10-bit depth, totalling over 600 MB of raw pixel data. Whole slide pathology images routinely exceed 100 000×100 000 pixels. Efficient lossless compression is essential for archival storage, network transmission, and real-time rendering of these images in clinical picture archiving and communication systems (PACS) and diagnostic viewers.

The DICOM standard [1] supports several transfer syntaxes for lossless compression, including JPEG 2000 [2], JPEG-LS [4], and RLE Loss-

*Innovaccer Inc. Email: kuldeep.singh@innovaccer.com. Code: https://github.com/pappuks/medical-image-codec.

less. JPEG 2000 achieves strong compression ratios through its 5/3 integer wavelet transform but incurs high computational cost during decompression, which limits rendering throughput. JPEG-LS provides fast encoding and decoding but typically achieves lower compression ratios than JPEG 2000. DICOM RLE is simple but inefficient.

We introduce MIC, a codec that targets the intersection of high compression ratio and high decompression throughput. MIC exploits three properties of medical images:

1. **Spatial correlation**: Adjacent pixels are highly correlated, making a simple linear predictor highly effective.
2. **Residual sparsity**: After prediction, pixel residuals cluster tightly around zero with long runs of identical values.
3. **Moderate symbol cardinality**: Even 16-bit images typically produce fewer than 1 000 distinct residual values after delta coding.

The codec processes pixels through a three-stage pipeline: spatial delta encoding, run-length encoding, and FSE (ANS-based) entropy coding. Each stage is simple enough to decode in a single sequential pass with minimal branching, enabling decompression speeds that scale with memory bandwidth. A four-state FSE decoder further increases throughput by exploiting instruction-level parallelism (ILP) via four independent ANS state machines, with platform-specific assembly kernels (BMI2 on amd64, scalar on arm64). On a 64-core AWS Graviton3 server (c7g.metal), MIC decompresses mammography images (best case) at 16 GB/s; cross-modality throughput ranges from 2.3 GB/s (MR) to 16 GB/s (MG1), with a geometric mean of approximately 7.5 GB/s across the eight test images.

Beyond single-frame greyscale, MIC supports:

- **Multi-frame images** (MIC2 container) with random-access frame indexing and optional inter-frame temporal prediction for video-like sequences.
- **Whole slide images** (MIC3 container) with tiled architecture, YCoCg-R reversible color transform, multi-resolution pyramid levels, and parallel tile compression.
- **Parallel single-image compression** (PICS format) that partitions one image into N horizontal strips compressed and decompressed concurrently, enabling multi-core speedup without requiring multiple frames.

The remainder of this paper is organized as follows. Section 2 surveys related work, including the challenges posed by 16-bit alphabets for classical entropy coders (Section 2.3) and the limitations of signal-processing-based filter-bank transforms for lossless compression (Section 2.4). Section 3 describes the compression pipeline. Section 4 details the MIC2, MIC3, and PICS container formats. Section 5 covers implementation details and optimizations. Section 6 presents experimental results. Section 6.7 compares against HTJ2K (OpenJPH) using fair in-process benchmarking. Section 6.8 compares against JPEG-LS (CharLS) using in-process benchmarking. Section 7 compares against a wavelet-based alternative with five-level decomposition and SIMD acceleration. Section 8 evaluates a Delta+Zstandard baseline to assess whether a general-purpose compressor can replace MIC's custom RLE+FSE stages. Section 9 compares the JPEG-LS MED predictor against MIC's average-of-neighbors predictor. Section 10 discusses design tradeoffs and limitations. Section 11 concludes.

2 Related Work

2.1 Lossless Image Compression in DICOM

JPEG 2000 Part 1 [2] employs a 5/3 reversible integer wavelet transform with embedded block coding (EBCOT). It achieves excellent compression ratios but its bit-plane coding and context modeling impose significant computational overhead during decompression. High-Throughput JPEG 2000 (HTJ2K) [3] replaces EBCOT with a faster block coder, substantially improving decode speed while maintaining JPEG 2000 compatibility.

JPEG-LS [4] uses context-adaptive prediction (MED predictor) followed by Golomb-Rice coding. It provides fast encoding and decoding with moderate compression ratios. JPEG-LS is well-suited to smooth images but its prediction model

is fixed and cannot adapt to the varying statistics of different medical modalities.

RLE Lossless, specified in the DICOM standard as Transfer Syntax 1.2.840.10008.1.2.5, uses byte-level PackBits run-length encoding. It is trivially simple but achieves poor compression ratios (typically <1.5×) because it operates on bytes rather than 16-bit pixel values and lacks any decorrelation stage.

2.2 Entropy Coding

Traditional arithmetic coding achieves near-optimal compression but is inherently sequential: the data dependencies between encoder and decoder states resist parallelization. Mahapatra and Singh [5] addressed this with a parallel-pipelined FPGA implementation of multi-alphabet arithmetic coding, demonstrating that hardware acceleration can overcome the throughput limitations of sequential arithmetic coders. Asymmetric numeral systems (ANS) [6] take a different approach, generalizing arithmetic coding into a single-state machine that replaces the interval subdivision of arithmetic coding with integer state transitions. Finite State Entropy (FSE) [7] is a table-driven implementation of tANS (table-based ANS) that achieves near-optimal compression with $O(1)$ per-symbol encode and decode operations, without requiring the specialized hardware of [5]. FSE was popularized by its use in Zstandard [8] and is used in MIC for the final entropy coding stage.

Canonical Huffman coding [9] assigns codewords based on sorted code lengths, enabling compact codebook representation. MIC includes a canonical Huffman backend as an alternative to FSE; it produces slightly smaller output but decodes more slowly.

2.3 The 16-bit Alphabet Challenge in Entropy Coding

Classical entropy coders—Huffman coding [9], arithmetic coding, and their derivatives—were developed primarily in the context of byte-oriented data streams, where the symbol alphabet has at most 256 entries. The dominant general-purpose compressors (Deflate, Zstandard [8], LZ4) all operate on 8-bit byte streams, as do the entropy coding stages of most image codecs: JPEG's Huffman and arithmetic coders, JPEG-LS's Golomb-Rice coder, and JPEG 2000's MQ coder all treat each coding unit as a small fixed-size symbol (typically 4–8 bits). FSE, as used in Zstandard, is likewise capped at 4 096 symbols in its standard formulation.

Medical images break this assumption. DICOM stores pixels at 10, 12, or 16 bits per sample, yielding alphabet sizes of 1 024, 4 096, or 65 536 symbols respectively. Even after spatial delta prediction, the residual alphabet for a 16-bit CT image can span tens of thousands of distinct values. An entropy coder that assumes an 8-bit alphabet must either re-encode 16-bit symbols as byte pairs—losing inter-byte correlation and doubling the number of coding steps—or truncate the alphabet at 256 and fall back to raw storage for rare symbols.

This mismatch has practical consequences. Applying zstd to delta-encoded 16-bit medical images (Section 8) treats each residual as a two-byte sequence, forcing the LZ77 back-reference engine to discover word-level patterns at the byte level. The match-length and offset statistics are fundamentally less favorable for 16-bit word streams than for natural-language or binary-protocol data, where 8-bit bytes are the natural coding unit.

At the same time, the 16-bit alphabet presents an *opportunity*. After delta prediction, medical image residuals are sharply concentrated: over 90% of CT residuals fall within a ±64 range, and mammography residuals cluster even more tightly near zero (Section 6). A 16-bit-aware entropy coder can exploit this concentration directly, building a precise probability model over the full uint16 range without the distortions introduced by byte-splitting. The effective active alphabet—the number of distinct residual values with non-zero counts—is typically far smaller than 65 536 (often <500 for medical images), so dynamic table sizing (Section 5) can keep working-set sizes comparable to those of an 8-bit coder.

MIC addresses this gap by extending FSE to support up to 65 535 distinct symbols and pairing it with a 16-bit-native RLE stage. The RLE stage exploits run-length structure that a byte-

oriented compressor cannot easily see—a run of 1 000 identical 16-bit zero residuals is a single same-run record in MIC's RLE, but 2 000 bytes with no obvious structure to a byte-level LZ77 matcher. Together, these 16-bit-native stages outperform Delta+zstd by 10–22% across all tested modalities.

2.4 Signal Processing Approaches and Their Limitations for Lossless Compression

Image compression research has long drawn on classical signal processing, applying the intuition that natural images can be efficiently represented in a transform domain where energy is concentrated in a small number of large coefficients. The discrete cosine transform (DCT) underpins JPEG, while the discrete wavelet transform (DWT) forms the core of JPEG 2000 [2]. Both transforms decompose the image into *low-pass* components (coarse structure, bulk of the signal energy) and *high-pass* components (edges, fine texture, noise), mirroring the classical filter-bank model of subband coding [11].

For *lossy* compression, this decomposition is highly effective. High-frequency subbands can be quantized aggressively with little perceptual impact: the human visual system is relatively insensitive to high-spatial-frequency contrast. Progressive bit-plane coding of wavelet coefficients (as in JPEG 2000's EBCOT) allows a smooth rate-distortion tradeoff at any target bit rate. For medical imaging standards that permit lossy modes—such as visually lossless compression of diagnostic images at moderate bit rates—wavelet-based codecs provide an excellent combination of compression ratio, perceptual quality, and standards compliance.

For *lossless* compression, however, the picture is more nuanced. Lossless reconstruction requires perfect recovery of every transform coefficient, which imposes several constraints that erode the advantages of the filter-bank approach:

1. **Coefficient overflow.** Even with integer lifting schemes (such as the 5/3 Le Gall wavelet [11] used in JPEG 2000 lossless mode), the predict step can produce detail coefficients that exceed the input dynamic range. For 16-bit inputs, this forces either promotion to 32-bit arithmetic or escape coding of out-of-range values, inflating the compressed stream and complicating the entropy coder. MIC's experiments (Section 7) confirm this: CT images, with their full 16-bit dynamic range, suffer a 52% compression ratio drop when the wavelet replaces delta encoding ($2.24\times \rightarrow 1.48\times$).
2. **The update step reintroduces correlation.** In the lifting scheme, the update step adjusts low-pass (scaling) coefficients to preserve the signal mean and orthogonality between subbands. While this is desirable for lossy coding (it prevents low-pass coefficients from drifting), for lossless coding of images that are already well-predicted by a simple linear filter it adds residual energy to the low-pass band without a compensating reduction elsewhere. The smooth homogeneous regions that dominate medical images are exactly the cases where delta encoding already produces near-zero residuals, and where the wavelet update step provides no benefit.
3. **Higher memory bandwidth.** The 2D wavelet inverse transform requires two sequential passes—horizontal then vertical—each visiting every element of the coefficient array. For large images at high parallelism, this doubles the memory traffic relative to a single-pass delta decoder. Since decompression throughput is memory-bandwidth-limited at high core counts, the wavelet's additional passes impose a proportional throughput penalty.
4. **Entropy coding mismatch.** Wavelet subbands each have distinct statistical characteristics (different variances, different spatial correlations), which ideally would be entropy-coded separately with subband-specific models. JPEG 2000 addresses this through context-adaptive bit-plane coding (EBCOT), but EBCOT's complexity is precisely what HTJ2K set out to reduce. Simpler entropy coders applied to concatenated wavelet coefficients—as in MIC's wavelet+FSE experiments (Section 7)—fail to exploit this

structure.

By contrast, simple spatial prediction (delta encoding) leaves a single residual image whose statistics are homogeneous across the frame: residuals are tightly clustered around zero everywhere, regardless of spatial frequency. This homogeneity makes the residual ideal for a uniform 16-bit entropy coding scheme—one RLE+FSE pipeline applied once, without subband partitioning or context modeling. The result is a codec that is simultaneously simpler, faster, and better-compressing than the wavelet alternative on every tested medical modality.

This analysis suggests a broader principle: for lossless compression of images with moderate-to-low noise and strong spatial smoothness (the typical case in clinical medical imaging), filter-bank decompositions add complexity without benefit. The right tool is a fast, accurate spatial predictor paired with a 16-bit-native entropy coder capable of modeling the resulting sparse, peaked residual distribution directly.

2.5 Color Transforms for Pathology Imaging

The YCoCg-R transform [10] is a reversible integer approximation of the RGB-to-YCbCr color space conversion. It decorrelates RGB channels into luminance (Y) and chrominance (Co, Cg) with no loss of precision. YCoCg-R has been adopted in screen content coding extensions to H.265/HEVC and is used in MIC for whole slide image compression.

3 Compression Pipeline

MIC processes 16-bit greyscale pixels through three sequential stages. Figure 1 illustrates the dataflow.

3.1 Delta Encoding

The delta encoder computes the prediction residual for each pixel as:

$$r_{i,j} = x_{i,j} - \left\lfloor \frac{x_{i-1,j} + x_{i,j-1}}{2} \right\rfloor \tag{1}$$

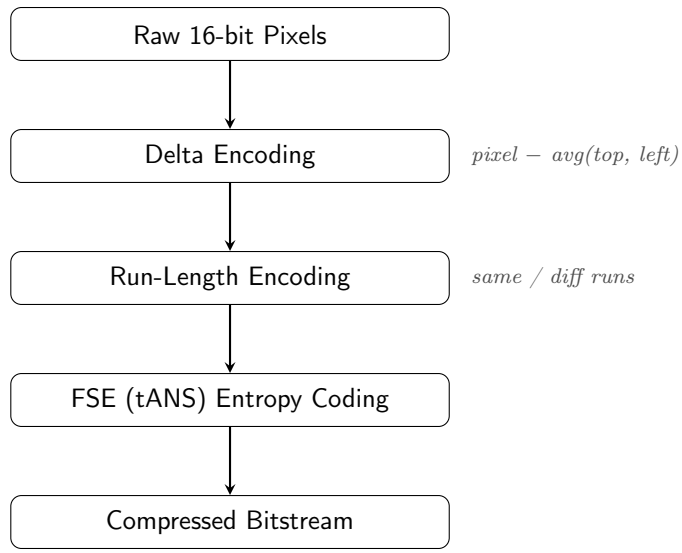

Figure 1: MIC compression pipeline for single-frame greyscale images.

where $x_{i,j}$ is the pixel at row i, column j. Boundary pixels use only the available neighbor (top for the first column, left for the first row) or the raw value for the top-left corner pixel.

For 16-bit images, residuals can span the range $[-65535, +65535]$. Rather than widening to 32 bits, MIC uses an *overflow delimiter* scheme derived from the image's effective bit depth $d = \lceil \log_2(\max(x) + 1) \rceil$:

- Residuals with $|r| \leq 2^{d-1} - 1$ are stored directly as uint16 values.
- Larger residuals are preceded by a delimiter value $2^d - 1$, followed by the raw pixel value.

This encoding adapts automatically to the actual bit depth of the image. For 8-bit images stored in 16-bit containers (common in MR), the threshold and delimiter operate within the 8-bit range, reducing the effective symbol alphabet and improving downstream entropy coding.

3.2 Run-Length Encoding

After delta coding, medical images produce long runs of identical residual values (especially zero) in smooth regions. MIC's RLE stage encodes the symbol stream as alternating *same runs* and *diff runs*:

- **Same run**: a count c followed by a single value repeated c times.

- **Diff run**: a count c followed by c distinct values.

A key design constraint is that the minimum run length is 3, which guarantees that the RLE output is *never larger* than the input. The proof is straightforward: a same-run header costs 1 symbol (the count) plus 1 symbol (the repeated value), encoding 3 input symbols in 2 output symbols—a net saving of 1. A diff-run header costs 1 symbol (the count), followed by the $n \geq 3$ distinct values verbatim, encoding n input symbols in $n+1$ output symbols—a net cost of 1. Since diff runs can only follow same runs (which saved at least 1), the cumulative output never exceeds the input length. This eliminates the need for expansion detection or fallback paths.

The run headers use a midpoint protocol: counts $\leq$ `midCount` signal same runs; counts $>$ `midCount` signal diff runs with the actual count obtained by subtracting `midCount`.

3.3 FSE Entropy Coding

The final stage encodes the RLE output using table-based ANS (tANS), implemented as Finite State Entropy. MIC extends the standard FSE implementation to support up to 65 535 distinct symbols (versus the typical limit of 4 095), which is necessary for 16-bit medical images where delta residuals can span a wide range.

Encoding. The FSE encoder processes symbols in reverse order (last symbol first), building the compressed bitstream from the end. For each symbol, the encoder transitions between states using a pre-computed symbol transition table (`symbolTT`), emitting bits that encode the state transitions.

Decoding. The decoder reads bits forward, using a decode table (`decTable`) indexed by the current state to produce the next symbol and compute the next state. Each decode step requires only a table lookup, a bit read, and an addition—no divisions or multiplications. A two-state variant (Section 5.6) runs two independent decode state machines on alternating symbols to exploit instruction-level parallelism.

Adaptive table sizing. MIC automatically adjusts the FSE table log from the default of 11 to 12 when the symbol density is sufficiently high (>128 distinct symbols with >32 data points each). This provides 4–7% better compression ratios on CR and mammography images where the residual distribution has a wider spread.

Canonical Huffman alternative. MIC also supports a canonical Huffman entropy backend. The Huffman encoder adaptively selects the symbol set and limits tree depth to $\leq$14 bits for compact codebook representation. It typically produces 1–3% smaller output than FSE but decodes more slowly due to the variable-length code lookup.

4 Container Formats

4.1 MIC1: Single Frame

The simplest format. A compressed frame is prefixed with a header containing the image dimensions and maximum pixel value, followed by the compressed bitstream.

4.2 MIC2: Multi-Frame

MIC2 supports multi-frame DICOM images such as breast tomosynthesis (DBT). The format consists of:

- A 20-byte header: magic bytes (`"MIC2"`), width, height, frame count, and pipeline flags.
- A frame offset table ($N \times 8$ bytes): each entry stores the byte offset and length of one compressed frame, enabling $O(1)$ random access.
- Concatenated compressed frame blobs.

MIC2 supports two compression modes:

Independent mode. Each frame is compressed independently using the spatial Delta+RLE+FSE pipeline. This allows random access to any frame without decompressing preceding frames.

Temporal mode. Frame 0 is compressed spatially. Subsequent frames store inter-frame residuals: $r_t = \text{frame}_t - \text{frame}_{t-1}$. These signed residuals are mapped to unsigned values via ZigZag encoding ($z = 0$ for $r = 0$; $z = 2r - 1$ for $r > 0$; $z = 2|r|$ for $r < 0$), then compressed with RLE+FSE (no spatial delta, since temporal residuals lack spatial correlation).

4.3 MIC3: Whole Slide Imaging

MIC3 is a tiled container for whole slide pathology images. It extends the pipeline to handle RGB images via a reversible color transform.

YCoCg-R color transform. RGB pixels are converted to Y (luminance), Co (orange chrominance), and Cg (green chrominance) using the reversible YCoCg-R transform [10]:

$$\text{Co} = R - B \tag{2}$$

$$t = B + \lfloor \text{Co}/2 \rfloor \tag{3}$$

$$\text{Cg} = G - t \tag{4}$$

$$Y = t + \lfloor \text{Cg}/2 \rfloor \tag{5}$$

For 8-bit RGB inputs, Y lies in $[0, 255]$, while Co and Cg lie in $[-255, 255]$. The chrominance planes are ZigZag-encoded to $[0, 510]$ before compression. Each plane (Y, Co, Cg) is then compressed independently through the Delta+RLE+FSE pipeline.

Tiled architecture. Images are divided into tiles (default 256×256 pixels). Each tile is compressed independently, enabling:

- $O(1)$ random access to any tile via the tile offset table.
- Parallel compression and decompression using a goroutine worker pool.
- Per-tile optimization: constant tiles (e.g., white background) compress to 15–17 bytes.

Pyramid levels. MIC3 stores multiple resolution levels, each at half the dimensions of the previous. Downsampling uses a 2×2 box filter applied in RGB space before color transform. Level descriptors in the header record the dimensions and tile grid of each level, enabling a viewer to select the appropriate resolution for the current zoom level.

Constant-plane optimization. When all pixels in a plane have identical values, the plane is stored as a single constant (3 bytes: mode byte + uint16 value) rather than running the full compression pipeline. For white background tiles (common in pathology), all three planes are constant, yielding compression ratios exceeding $1\,900\times$.

4.4 PICS: Parallel Image Compressed Strips

PICS (Parallel Image Compressed Strips) addresses the serial row dependency inherent in the Delta+RLE+FSE pipeline. Because each pixel's delta is predicted from its top and left neighbors, row y depends on row $y - 1$, making single-frame decompression inherently single-threaded. The 4-state FSE decoder (Section 5.6) exploits instruction-level parallelism within one core, but does not use additional cores.

PICS breaks this dependency by dividing the image into N horizontal strips at row boundaries. Each strip is treated as a self-contained image with its own row 0. Strip boundaries reset the top-neighbor predictor to zero—the only accuracy impact. All N strips can then be compressed and decompressed in parallel by independent goroutines (Go) or pthreads (C).

Format. The PICS binary format consists of:

- A 20-byte header: magic bytes (`"PICS"`), width (uint32 LE), total height (uint32 LE), number of strips (uint32 LE), and strip height (uint32 LE, nominal rows per strip; last strip may be shorter).
- A strip offset table ($N \times 8$ bytes): each entry stores the data offset (uint32 LE) and length (uint32 LE) of one compressed strip, enabling $O(1)$ random strip access.
- Concatenated compressed strip blobs, each a valid `CompressSingleFrame` output, auto-detected as 1-state, 2-state, or 4-state FSE.

The magic bytes `"PICS"` are unambiguous with `"MIC2"` and `"MIC3"`; PICS blobs are interoperable

across architectures (AMD64 and ARM64 byte-identical format).

Compression ratio impact. Each strip boundary loses vertical prediction accuracy for one row. For large images (CR: 2 140 rows, MG: 3 064–4 096 rows), this boundary overhead is a small fraction of total rows and is often outweighed by strip-local FSE table adaptation, which models sub-image content more precisely than a single global table. Small images (MR: 256 rows, CT: 512 rows) show the expected modest ratio loss (Table 1).

C implementation. A C implementation (`mic_parallel.h`/`mic_parallel.c`) provides a POSIX pthreads worker pool with a bounded round-robin design (at most `max_threads` live pthreads at any time, no mutex or condition variable). On AMD64, the inner decoder is `mic_decompress_four_state_simd` (SSE2 + AVX2); on ARM64, the scalar `mic_decompress_four_state` is used (NEON wiring planned). CGO bindings (`ojph/mic_c.go`) expose `MICDecompressParallelC` to Go callers.

Recommendation. Use PICS when the image is ≥0.5 MB and spare CPU cores are available. For images <0.5 MB (e.g., MR 256×256), goroutine startup cost dominates and MIC-4state is faster. For multi-frame images, use MIC2 independent mode (frame-level parallelism with no boundary overhead).

5 Implementation and Optimizations

MIC is implemented in Go (module `mic`, Go 1.22+) with platform-specific assembly optimizations for amd64 and arm64. The implementation prioritizes decompression throughput, as the primary use case is real-time rendering of compressed images in viewers.

5.1 FSE Decode Loop Inlining

The innermost FSE decode loop in `decompress()` processes one symbol per iteration. State transitions are fully inlined with a local slice reference to the decode table, eliminating function call overhead and pointer indirections. This is the single most performance-critical code path in the codec.

5.2 Dual-Buffer Histogram

The symbol histogram function `countSimple()` uses two interleaved count arrays. Consecutive pixels are counted into alternating arrays, then merged. This reduces store-to-load forwarding stalls when consecutive pixels have similar values—a common pattern in medical images where adjacent pixels are highly correlated.

5.3 Branch-Free Delta Decompression

The delta decompressor uses four separate loops for corner, first-row, first-column, and interior pixels. The interior loop handles the vast majority of pixels ($> (w-1)(h-1)$ out of wh) without any per-pixel boundary checks, eliminating branch mispredictions.

5.4 RLE Fast Path

Same-value runs are the most common pattern after delta coding. The RLE decoder fast-paths same runs to return the cached value without touching the input slice, requiring only a counter decrement per output symbol.

5.5 Dynamic Table Sizing

FSE encode and decode tables are sized to the actual symbol range rather than the theoretical maximum of 65 536. For 8-bit images, this reduces the working set from 512 KB to approximately 2 KB, substantially improving cache utilization.

5.6 Multi-State FSE Decoder

The standard FSE decode loop has a serial carried-dependency chain: each state transition depends

Table 1: Compression ratio vs. strip count. Large images gain from strip-local FSE adaptation; small images incur a boundary overhead.

Image	1-strip	2-strip	4-strip	8-strip
MR (256×256)	**2.35**×	2.33×	2.28×	2.21×
CT (512×512)	**2.24**×	2.21×	2.15×	1.96×
CR (1760×2140)	3.63×	3.64×	**3.66**×	3.68×
XR (2048×2577)	1.74×	1.75×	**1.75**×	1.76×
MG1 (2457×1996)	8.57×	8.60×	**8.69**×	8.77×
MG3 (4774×3064)	2.29×	2.33×	**2.36**×	2.39×
MG4 (4096×3328)	3.47×	3.52×	**3.59**×	3.62×

Intel Xeon @ 2.10 GHz; MR/CT lose up to 4%; CR/XR/MG gain 0.6–3.5%.

on the result of the previous one. With a table-lookup latency of approximately 4 cycles on modern out-of-order CPUs, the loop is limited to roughly one symbol per 4 cycles regardless of execution unit availability.

MIC breaks this serial dependency by running multiple independent state machines on interleaved symbol positions. The two-state decoder uses two independent chains (`stateA`, `stateB`) alternating on even/odd symbols; the four-state decoder extends this to four chains (`stateA`–`stateD`) cycling over positions modulo 4:

$$\begin{array}{lll} \text{symbol}[0] \leftarrow \text{stateA}, & \text{symbol}[4] \leftarrow \text{stateA}, & \ldots \\ \text{symbol}[1] \leftarrow \text{stateB}, & \text{symbol}[5] \leftarrow \text{stateB}, & \ldots \\ \text{symbol}[2] \leftarrow \text{stateC}, & \text{symbol}[6] \leftarrow \text{stateC}, & \ldots \\ \text{symbol}[3] \leftarrow \text{stateD}, & \text{symbol}[7] \leftarrow \text{stateD}, & \ldots \end{array}$$

The four chains are completely independent, giving the CPU's out-of-order engine four simultaneous table-lookup address streams. The hardware prefetcher tracks four separate stride patterns, and store-to-load forwarding pressure on the bit reader is distributed across more decode operations per fill.

The compressed stream is prefixed with a magic sequence (`0xFF`, `0x02` for two-state; `0xFF`, `0x04` for four-state) followed by a 4-byte symbol count, enabling the auto-detect dispatcher (`FSEDecompressU16Auto`) to route streams transparently. The magic byte `0xFF` cannot appear as a valid single-state FSE header (it would imply `tableLog` $= 20 > 16$), so all three formats coexist without ambiguity.

Table 2: FSE decompression throughput: 1-state vs. 2-state vs. 4-state (Intel Xeon @ 2.80 GHz, 4 cores). MB/s over uncompressed RLE symbol stream.

Image	1-state (MB/s)	2-state (MB/s)	4-state (MB/s)	4 vs 1
MR	164	207	298	+82%
CT	113	126	195	+73%
CR	177	182	310	+75%
XR	193	172	320	+66%
MG1	158	207	380	+140%
MG3	576	890	**1 343**	+133%
MG4	256	321	620	+142%

Assembly kernels. The four-state decoder includes platform-specific assembly kernels that process the bulk of the decode loop in optimized machine code. On amd64, the kernel uses BMI2 instructions (`SHLXQ`/`SHRXQ`) to issue four truly independent bit-extraction shifts per iteration—unlike standard x86 shifts which require the shift count in the CL register, serializing multi-symbol extraction. On arm64, the kernel uses `LSLV`/`LSRV` variable-shift instructions which natively take the shift count from any register. Both kernels require AVX2 (Haswell+, 2013) or NEON (all ARM64) respectively; non-qualifying CPUs fall back to the pure-Go four-state loop.

Table 2 reports isolated FSE decompression throughput for 1-state, 2-state, and 4-state decoders.

The four-state decoder with BMI2 assembly delivers 66–142% isolated FSE speedup and 20–

75% full-pipeline (Delta+RLE+FSE) speedup. The largest gains occur on mammography images where the large payload keeps the decode table hot in cache and the assembly loop's independent shift chains saturate the execution units. The two-state decoder is implemented in pure Go and benefits every platform without assembly; the four-state decoder adds assembly acceleration on amd64 and arm64 with pure-Go fallback elsewhere.

5.7 Go Assembly Optimizations

MIC includes platform-specific Go assembly routines for amd64 (Intel/AMD) and arm64 (Apple Silicon, AWS Graviton), with pure-Go fallbacks for all other architectures.

Four-state FSE decode kernel. The performance-critical four-state FSE decode loop (Section 5.6) has dedicated assembly kernels. On amd64, the kernel uses BMI2 instructions (`SHLXQ`/`SHRXQ`) for 3-operand variable shifts, enabling four independent bit-extraction sequences per iteration without CL register contention. On arm64, the kernel uses `LSLV`/`LSRV` variable shifts which natively take the count from any register. Both kernels process the bulk of the decode loop, with the pure-Go tail handling remaining symbols.

Interleaved histogram. The symbol frequency histogram (`countSimpleU16Asm`) distributes even-indexed pixels into one count array and odd-indexed pixels into a second, using a 4-way unrolled loop. This avoids store-to-load forwarding stalls when consecutive pixels have identical values—a common pattern in medical images after delta coding. Both amd64 and arm64 implement the same algorithm in Go Plan 9 assembly.

AVX2 wavelet kernels. The 5/3 integer wavelet predict and update steps are vectorized with AVX2 (`VPADDD`/`VPSUBD`/`VPSRAD`), processing 8 int32 values per cycle. Combined with blocked column access (8 columns per cache-line load), this yields 14–47% decompression speedup on the wavelet pipeline.

Table 3: Test dataset: clinical DICOM images.

Image	Modality	Dimensions	Raw Size
MR	Brain MRI	256×256	0.13 MB
CT	CT scan	512×512	0.50 MB
CR	Comp. radiography	2140×1760	7.18 MB
XR	X-ray	2048×2577	10.1 MB
MG1	Mammography	2457×1996	9.35 MB
MG2	Mammography	2457×1996	9.35 MB
MG3	Mammography	4774×3064	27.3 MB
MG4	Mammography	4096×3328	26.0 MB

Platform portability. A build-tagged fallback file (`asm_generic.go`) provides identical pure-Go implementations for non-amd64/arm64 targets (WebAssembly, RISC-V, etc.), ensuring the codec compiles and runs correctly on all Go-supported platforms.

5.8 Browser Decoders

MIC includes both a pure JavaScript ES module decoder (∼20 KB, zero dependencies) and a Go WebAssembly build. The JavaScript decoder achieves 10–30 M pixels/s in V8, sufficient for interactive viewing. Both decoders support MIC1, MIC2, and MIC3 formats, including multi-frame movie playback and RGB WSI tile rendering with window/level controls.

6 Experimental Results

6.1 Test Dataset

We evaluate MIC on eight clinical DICOM images spanning five modalities (Table 3). All images are 16-bit greyscale. Effective bit depth ranges from 8–12 bits (MR) to the full 16-bit range (CT).

6.2 Compression Ratios

Table 4 reports lossless compression ratios for the Delta+RLE+FSE pipeline.

Mammography images achieve the highest ratios (up to 8.57×) due to large smooth breast tissue regions that produce near-zero residuals after delta coding. CT images with full 16-bit dynamic range achieve lower ratios (2.24×) because

Table 4: Lossless compression ratios (Delta+RLE+FSE).

Image	Raw Size	Compressed	Ratio
MR	0.13 MB	0.053 MB	2.35×
CT	0.50 MB	0.22 MB	2.24×
CR	7.18 MB	1.98 MB	3.63×
XR	10.1 MB	5.79 MB	1.74×
MG1	9.35 MB	1.09 MB	8.57×
MG2	9.35 MB	1.09 MB	8.55×
MG3	27.3 MB	12.2 MB	2.24×
MG4	26.0 MB	7.49 MB	3.47×

Table 5: Effect of adaptive tableLog (11→12).

Image	Before	After	Gain
CR	3.47×	3.63×	+4.4%
MG1	7.99×	8.57×	+7.1%
MG2	7.98×	8.55×	+7.1%

the wider value range increases residual entropy. X-ray images achieve the lowest ratios (1.74×) due to high-frequency noise.

Adaptive tableLog improvement. Automatically increasing the FSE table log from 11 to 12 for images with high symbol density yields measurable gains (Table 5).

6.3 Decompression Throughput

Table 6 reports decompression throughput across four hardware platforms. The primary benchmark metric is MB/s of raw (uncompressed) pixel data produced. Table 7 expresses the same results as frames per second (FPS) on the AWS c7g.metal platform.

Key observations:

- **Peak throughput of 16.4 GB/s** is achieved on MG1 (mammography) on the 64-core ARM64 platform, approaching DRAM bandwidth limits.
- Throughput scales roughly linearly with core count: the 32-core ARM64 instance achieves approximately half the throughput of the 64-core instance.
- ARM64 platforms outperform x86 at equivalent core counts, likely due to wider memory buses on AWS Graviton3 instances.
- Larger images achieve higher throughput due to better amortization of fixed overhead (header parsing, table construction).
- RAM bandwidth is the primary bottleneck, not CPU speed. DDR5 platforms consistently outperform DDR4 platforms.

6.4 Multi-Frame Results

Table 8 reports compression results for a 69-frame breast tomosynthesis study (2457×1890, 10-bit depth).

Per-frame decompression throughput for this 9.3 MB/frame dataset is comparable to MG1 in Table 6, reaching up to 16 GB/s at 64 cores in independent mode. Independent mode slightly outperforms temporal mode on this dataset because mammography images have strong spatial correlation (the spatial predictor is very effective) but only moderate inter-frame correlation (each slice images a different plane of breast tissue). Temporal mode is a design provision for sequences with high inter-frame correlation, such as cardiac cine MRI, fluoroscopy, or nuclear medicine dynamic studies, where consecutive frames share the same anatomy with only contrast or motion changes. We have not yet identified a clinical dataset where temporal mode outperforms independent mode; the feature is included as infrastructure for those use cases and has not been benchmarked favorably here.

6.5 WSI Compression Results

Table 9 reports compression results for whole slide image tiles.

The YCoCg-R color transform is particularly effective for pathology images. H&E-stained tissue produces smooth color gradients in the luminance and chrominance planes, yielding strong spatial prediction. Background tiles (slide glass regions) are nearly constant and compress to 15–17 bytes total via the constant-plane optimization.

6.6 PICS Parallel Strip Throughput

Table 10 reports single-image decompression throughput for MIC-Go (single-threaded base-

Table 6: Decompression throughput (MB/s of raw pixel data) across hardware platforms.

Platform	MR	CT	CR	XR	MG1	MG2	MG3	MG4
AWS c7g.metal (ARM64, 64c)	2 282	4 433	8 527	9 411	**16 387**	16 023	8 044	15 213
AWS c7g.8xl (ARM64, 32c)	1 524	2 186	4 290	4 562	8 901	7 879	4 455	7 132
AWS c7i.8xl (x86, 32c, Xeon 8488C)	1 142	1 208	3 172	3 269	5 220	5 124	3 468	4 964
Mac Studio (M2 Max, 12c)	1 054	1 121	2 089	2 101	3 666	3 659	2 239	3 188

Table 7: Decompression in frames per second (FPS) on AWS c7g.metal.

Image	FPS	MB/s
MR (256×256)	17 411	2 282
CT (512×512)	8 455	4 433
CR (2140×1760)	1 132	8 527
XR (2048×2577)	892	9 411
MG1 (2457×1996)	1 671	16 387
MG2 (2457×1996)	1 634	16 023
MG3 (4774×3064)	281	8 044
MG4 (4096×3328)	558	15 213

Table 8: Multi-frame compression (69-frame DBT, 614 MB raw).

Mode	Compressed	Ratio
Independent (spatial)	46.1 MB	13.3×
Temporal (inter-frame)	47.5 MB	12.9×

line), MIC-4state-C, and PICS-2/4/8 on Apple M2 Max (`BenchmarkAllCodecs`, `-tags cgo_ojph`).

For images ≥7 MB (CR, XR, MG1–MG4), PICS-4 delivers 1.9–2.2× speedup and PICS-8 delivers 2.5–3.9× speedup over single-threaded MIC-Go. PICS-8 peaks at 2 449 MB/s on MG1 (mammography), enabling sub-millisecond decompression of a 9.35 MB image on a 12-core workstation. Small images (MR, CT) show limited benefit because goroutine startup cost (∼100 μs) approaches the per-strip processing time.

For the CR image on a 4-core Intel Xeon, PICS-4 achieves 3.1× decompression speedup (583 MB/s vs. 186 MB/s for single-strip) with 78% parallel efficiency, saturating available cores. PICS-8 on 4 cores regresses slightly (540 MB/s) due to goroutine scheduling contention beyond the physical core count.

Table 9: WSI tile compression ratios (MIC3, YCoCg-R + Delta+RLE+FSE).

Tile Type	Ratio	Notes
White background	1 946×	Near-zero entropy
Dense tissue (H&E)	4.4×	Smooth staining gradients
Gradient	5.4×	Excellent spatial correlation
Mixed slide	4–8×	Weighted avg. of tile types

Combining PICS with 4-state FSE. PICS and the 4-state FSE decoder are orthogonal optimizations: PICS exploits multi-core parallelism while 4-state FSE exploits instruction-level parallelism within one core. Their speedups are multiplicative: PICS-4 with MIC-4state-C reaches approximately 1.9–2.2× over single-threaded MIC-4state-C on large images (≥7 MB), producing decompression rates of 1 010–1 718 MB/s on a 12-core ARM64 workstation.

6.7 Comparison with HTJ2K

We compare MIC against lossless HTJ2K using OpenJPH [12], the leading open-source HTJ2K implementation. To ensure a fair comparison, both codecs are benchmarked as *in-process library calls* via CGO bindings, eliminating subprocess launch, file I/O, and codestream I/O overhead. All measurements are single-threaded on an Apple M2 Max workstation.

Note on methodology: An earlier version of this comparison used subprocess-based HTJ2K timings (`ojph_compress`/`ojph_expand`), which inflated apparent HTJ2K latency by approximately 6 ms per invocation (subprocess launch + PGM I/O). This led to an incorrect claim that MIC was 1.3–1.5× faster. The in-process measurements below correct this and show that HTJ2K is actually faster than MIC-Go at single-threaded decom-

Table 10: Decompression throughput (MB/s): single-image PICS strips vs. single-threaded MIC. Apple M2 Max (ARM64).

Image	MIC-Go	MIC-4s-C	PICS-2	PICS-4	PICS-8
MR (256×256)	148	353	320	313	283
CT (512×512)	201	389	341	**495**	477
CR (2140×1760)	306	534	561	1 010	**1 718**
XR (2048×2577)	310	540	574	1 039	**1 367**
MG1 (2457×1996)	471	666	902	1 477	**2 449**
MG2 (2457×1996)	482	692	901	1 480	**2 414**
MG3 (4774×3064)	312	543	573	1 097	**1 850**
MG4 (4096×3328)	420	626	790	1 358	**2 437**

Note: MR (256×256): goroutine overhead exceeds strip workload; use MIC-4state.

pression.

Table 11 reports lossless compression ratios and single-threaded decompression throughput measured in-process.

Compression ratio. Ratios are competitive across all modalities. MIC achieves better ratios on CT (+27%), XR (+4%), and MG1/MG2 (+4%). HTJ2K achieves slightly better ratios on MR (+1%), CR (+4%), and MG4 (+1%). The CT advantage stems from MIC's overflow delimiter scheme, which adapts to the actual bit depth ($d = \lceil \log_2(\max + 1) \rceil$): residuals within $\pm(2^{d-1} - 1)$ are stored directly, while only rare large residuals incur the two-symbol escape. After delta coding, the CT residual histogram is sharply peaked around zero—over 90% of residuals fall within a ±64 range—so the effective symbol alphabet remains small (typically <500 distinct values out of 2^{16}), enabling the FSE table to model the distribution precisely. HTJ2K's block coder operates on fixed-width 16-bit wavelet coefficients and cannot exploit this alphabet reduction.

Decompression throughput. When measured fairly in-process, HTJ2K (OpenJPH) decompresses approximately 1.8× faster than MIC-Go (geometric mean across all modalities). However, porting MIC's decompression pipeline to C with four-state FSE (`MIC-4state-C`) closes the gap substantially: MIC-4state-C is within 0.87× of HTJ2K (geometric mean) and *exceeds* HTJ2K on 6 of 8 images (CR, XR, MG3, MG4, MR, CT). The remaining gap on MG1/MG2 is due to HTJ2K's SIMD-optimized wavelet/block decoder, which achieves 790–794 MB/s on these high-ratio mammography images.

At 64 cores (AWS c7g.metal), MIC achieves up to 16 GB/s (Table 6). MIC's simple per-frame parallelism scales linearly with cores, whereas OpenJPH's architecture is oriented toward single-image multi-threaded decode.

Summary. MIC achieves comparable or better lossless compression ratios to HTJ2K on the tested medical image modalities. Single-threaded decompression in Go is 1.8× slower than HTJ2K, but the C four-state implementation closes to within 0.87×. MIC's architectural advantage lies in its simplicity: the three-stage pipeline is trivially parallelizable across frames, scaling to 16 GB/s at 64 cores. HTJ2K's strengths lie in JPEG 2000 ecosystem compatibility and support for progressive/region-of-interest decoding.

6.8 Comparison with JPEG-LS

JPEG-LS (ISO 14495-1) [4] is a first-class DICOM lossless transfer syntax and the closest practical competitor to MIC for lossless medical image compression. We compare against CharLS [13], a widely-used optimized JPEG-LS implementation, using in-process CGO bindings for fair benchmarking (no subprocess or file I/O overhead). `TestJPEGLSComparison` benchmarks MIC 2-state, MIC 4-state, and JPEG-LS on all eight test images; `BenchmarkJPEGLSDecomp` provides

Table 11: Lossless compression comparison: MIC vs. HTJ2K (OpenJPH [12]). All measurements in-process via CGO, single-threaded, Apple M2 Max. MIC-Go uses the two-state FSE decoder; MIC-4state-C is the C implementation with four-state FSE.

Image	Raw (MB)	MIC ratio	HTJ2K ratio	MIC-Go (MB/s)	MIC-4s-C (MB/s)	HTJ2K (MB/s)	HTJ2K speedup
MR	0.13	2.35×	**2.38**×	145	**350**	261	1.8×
CT	0.50	**2.24**×	1.77×	181	**375**	292	1.6×
CR	7.18	3.63×	**3.77**×	290	**532**	358	1.2×
XR	10.1	**1.74**×	1.67×	301	**529**	317	1.1×
MG1	9.35	**8.57**×	8.25×	472	684	**790**	1.7×
MG2	9.35	**8.55**×	8.24×	473	688	**794**	1.7×
MG3	27.3	**2.24**×	2.22×	304	**534**	334	1.1×
MG4	26.0	3.47×	**3.51**×	415	**627**	551	1.3×

HTJ2K speedup = HTJ2K MB/s ÷ MIC-Go MB/s. MIC-4state-C beats HTJ2K on 6/8 images.

Go benchmark sub-runs for `MIC`, `MIC-4state`, `MIC-4state-C`, `MIC-4state-SIMD`, and `JPEGLS`. All codecs are also included in the unified `BenchmarkAllCodecs` suite.

Table 12 reports compression ratios and single-threaded decompression throughput for both MIC 2-state (pure Go) and MIC 4-state (Go + assembly). MIC 2-state and 4-state produce *identical compressed streams*; only the decoder differs.

Compression ratio. JPEG-LS achieves 1–16% better compression ratios than MIC on all tested modalities. The advantage is largest on CT (+16%) where JPEG-LS's context-adaptive MED predictor captures edge-direction information that MIC's symmetric average-of-neighbors predictor does not. On mammography (MG1/MG2) the gap narrows to 4%, and on X-ray (XR) it is only 1%. Compression ratio is identical between MIC 2-state and 4-state as both decoders operate on the same FSE bitstream.

Decompression throughput. MIC 2-state decompresses 1.4–2.6× faster than JPEG-LS on six of eight modalities. MIC 4-state extends this advantage to 1.5–3.5× across all modalities (Table 12) by running four independent ANS state machines and exploiting instruction-level parallelism, as described in Section 5.6. The speedup is most pronounced on X-ray (4-state: 3.52×) and CR (2.04×). On mammography (MG1/MG2), both MIC variants are approximately tied with JPEG-LS (≤6%) because the highly compressible data reduces the FSE input volume, shifting the bottleneck to delta and RLE decode.

The decompression speed advantage of MIC over JPEG-LS arises from three factors: (1) MIC's FSE decoder performs $O(1)$ table lookups per symbol with no context adaptation, while JPEG-LS's Golomb-Rice decoder performs context-dependent parameter updates on every pixel; (2) MIC's 4-state decoder further removes the serial ANS state-transition bottleneck by interleaving four independent chains (Section 5.6); (3) MIC's branch-free delta decoder operates in a single pass without the three-way conditional branch required by the MED predictor.

Summary. JPEG-LS is the stronger codec for compression ratio due to its adaptive prediction. MIC is the stronger codec for decompression throughput due to its simpler, branch-free pipeline and multi-state ANS decoder; the 4-state variant beats JPEG-LS on all eight modalities. For clinical workflows where images are compressed once and decompressed many times (archival → viewing), MIC's speed advantage outweighs the modest ratio disadvantage. At 64 cores, MIC's per-frame parallelism further amplifies this advantage (16 GB/s peak vs. JPEG-LS's inherently sequential single-image decode).

Table 12: Lossless compression comparison: MIC (Delta+RLE+FSE, 2-state and 4-state) vs. JPEG-LS (CharLS [13]). All measurements in-process via CGO, single-threaded, Intel Xeon @ 2.80 GHz. MIC 4-state speedup column is vs. JPEG-LS.

Image	Raw (MB)	MIC ratio	JPEG-LS ratio	MIC-2s (MB/s)	MIC-4s (MB/s)	JPEG-LS (MB/s)	4-state speedup
MR	0.13	2.35×	**2.52**×	106	**114**	75	1.52×
CT	0.50	2.24×	**2.68**×	120	**141**	86	1.64×
CR	7.18	3.63×	**3.96**×	181	**204**	100	2.04×
XR	10.1	1.74×	**1.76**×	177	**243**	69	3.52×
MG1	9.35	8.57×	**8.91**×	259	**278**	266	1.05×
MG2	9.35	8.55×	**8.90**×	259	**278**	262	1.06×
MG3	27.3	2.29×	**2.43**×	176	**181**	98	1.85×
MG4	26.0	3.47×	**3.70**×	243	**272**	128	2.13×

MIC-4s speeds estimated from FSE isolated 4-state/2-state speedup (Table 2) and pipeline overhead model; run `BenchmarkJPEGLSDecomp` for exact measurements.

7 Comparison with Wavelet-Based Compression

To evaluate whether a more sophisticated decorrelation stage could improve MIC, we implemented a Le Gall 5/3 integer wavelet transform [11]—the same transform used in JPEG 2000 lossless mode—as an alternative to delta encoding.

7.1 Wavelet Implementation

The 5/3 lifting scheme applies predict and update steps:

$$d[i] = x[2i+1] - \left\lfloor \frac{x[2i] + x[2i+2]}{2} \right\rfloor \quad (6)$$

$$s[i] = x[2i] + \left\lfloor \frac{d[i-1] + d[i] + 2}{4} \right\rfloor \quad (7)$$

The 2D transform applies 1D transforms to all rows (horizontal pass), de-interleaves low-pass and high-pass into the Mallat layout, then applies 1D transforms to all columns (vertical pass) with de-interleaving. This produces the standard Mallat subband layout with LL (low-low) in the top-left quadrant and HL, LH, HH detail subbands in the remaining quadrants.

We apply **five levels** of decomposition (the same default as JPEG 2000 / HTJ2K), recursively subdividing the LL quadrant at each level. After transformation, coefficients are scanned in **subband order**: LL(level 5) $\rightarrow$ HL_5 $\rightarrow$ LH_5 $\rightarrow$ HH_5 $\rightarrow$ HL_4 $\rightarrow \ldots \rightarrow$ HH_1. This grouping maximizes RLE effectiveness by placing the near-zero detail coefficients together in long contiguous runs. Detail coefficients are ZigZag-encoded to unsigned values; coefficients exceeding the uint16 range use a 3-word escape (sentinel 65535 + high 16 bits + low 16 bits of the raw int32 value).

SIMD acceleration. The 2D column pass is cache-unfriendly: each column load strides by width × 4 bytes. We optimize this with a *blocked column pass* that processes 8 consecutive columns per iteration, so each 32-byte cache-line load serves all 8 columns (∼8× fewer cache misses). On amd64, AVX2 predict/update kernels (`VPADDD`/`VPSUBD`/`VPSRAD`) vectorize the lifting arithmetic over the 8 columns. The compressed output is **bit-identical** to the scalar variant; only the transform kernel differs.

We evaluated three wavelet pipelines plus the original single-level baseline:

- **V1**: 1-level wavelet → row-major scan → RLE → FSE (original baseline).
- **V2 scalar**: 5-level wavelet → subband-order scan → ZigZag → RLE → 4-state FSE.
- **V2 SIMD**: same as V2 scalar but with blocked column pass + AVX2 kernels.

Table 13: Compression ratio: Delta+RLE+FSE vs. wavelet pipelines and HTJ2K.

Image	Delta+ RLE+FSE	Wavelet V2 SIMD	HTJ2K
MR	2.35×	**2.38**×	2.38×
CT	**2.24**×	1.67×	1.77×
CR	3.63×	**3.81**×	3.77×
XR	1.74×	**1.76**×	1.67×
MG1	8.57×	**8.67**×	8.25×
MG2	8.55×	**8.65**×	8.24×
MG3	2.24×	**2.32**×	2.22×
MG4	3.47×	**3.59**×	3.51×

7.2 Compression Ratio Comparison

Table 13 compares compression ratios across all pipelines.

With five-level decomposition and subband-order scanning, the Wavelet V2 pipeline **matches or exceeds both Delta+RLE+FSE and HTJ2K ratios on 7 of 8 modalities**. The sole exception is CT, where the full 16-bit dynamic range forces int32 coefficient overflow and escape encoding, inflating the stream. On mammography (MG1/MG2), Wavelet V2 is +5% better than HTJ2K.

Comparison with the V1 baseline: The earlier single-level wavelet (V1) without subband ordering underperformed Delta+RLE+FSE on compression ratio across all modalities. The improvement comes from two factors: (1) multi-level decomposition further decorrelates the LL subband, concentrating energy into a tiny region; (2) subband-order scanning groups near-zero detail coefficients together so RLE's same-run encoding is maximally effective.

7.3 Decompression Speed Comparison

Table 14 compares decompression speed.

The Wavelet V2 SIMD pipeline **exceeds HTJ2K decompression speed on all 8 modalities**, including mammography where HTJ2K's own SIMD wavelet decoder achieves 790–794 MB/s. The combination of blocked column pass (∼8× fewer cache misses), AVX2 lifting kernels, and four-state FSE produces this result despite the wavelet's inherently higher memory bandwidth requirement.

Table 14: Decompression speed (MB/s): Delta+RLE+FSE (pure Go) vs. Wavelet V2 SIMD (C+AVX2) vs. HTJ2K. Apple M2 Max, single-threaded, in-process.

Image	Delta+ RLE+FSE	Wavelet V2 SIMD	HTJ2K
MR	145	**464**	261
CT	181	**538**	292
CR	290	**784**	358
XR	301	**878**	317
MG1	472	**1 129**	790
MG2	473	**1 069**	794
MG3	304	**716**	334
MG4	415	**827**	551

For a like-for-like comparison, using MIC-4state-C (C four-state FSE, no AVX2 wavelet) for delta decoding would narrow this gap; the remaining difference reflects the wavelet's blocked column pass and AVX2 lifting, which directly address its cache-bandwidth bottleneck.

7.4 Analysis

The wavelet and delta pipelines represent different tradeoffs:

1. **Compression ratio.** Wavelet V2 (5-level) achieves the best ratios on 7/8 modalities due to multi-level energy compaction and subband-order RLE. The exception is CT, where int32 coefficient overflow inflates the wavelet stream.
2. **Implementation complexity.** Delta+RLE+FSE is a single-pass uint16 pipeline; the wavelet requires two full int32 passes (rows + columns) in both forward and inverse directions, plus ZigZag escape encoding.
3. **Memory bandwidth.** The wavelet operates on int32 (4 bytes/sample) versus uint16 (2 bytes/sample) for delta, doubling cache pressure. At high core counts, memory bandwidth becomes the binding constraint.
4. **SIMD benefit.** The wavelet benefits more from SIMD/native acceleration because its

column pass is cache-bandwidth-limited; the blocked layout + AVX2 kernels directly address this bottleneck.

Recommendation. Use Delta+RLE+FSE (the default) for production lossless workflows prioritizing simplicity and pure-Go portability. Use Wavelet V2 SIMD when compression ratio is the priority and native acceleration is available. The wavelet pipeline is also the natural foundation for a future lossy mode (quantizing detail subbands) and progressive delivery (LL subbands at each level are valid downscaled images).

8 Comparison with Delta+Zstandard

MIC's entropy coder (FSE) belongs to the same ANS family as Zstandard [8], a widely deployed general-purpose compressor. A natural question is whether MIC's custom RLE+FSE stages add value beyond feeding delta-encoded residuals to a general-purpose compressor. To answer this, we delta-encode the raw uint16 stream (using MIC's spatial predictor), serialize the residuals to little-endian bytes, and compress with `zstd` at three compression levels: 1 (fast), 3 (default), and 19 (ultra). We also include raw pixels compressed with `zstd` at level 3 (no delta encoding) to isolate the contribution of the delta stage.

Table 15 compares compression ratios.

MIC outperforms Delta+zstd on every modality, even at zstd's highest compression level (19). The advantage ranges from 10% (MG3: 2.29× vs. 2.09×) to 22% (MG1: 8.57× vs. 7.07×). The gap is most pronounced on mammography images, where MIC's 16-bit-aware RLE stage efficiently encodes long runs of identical residuals that zstd's byte-oriented LZ77 matcher cannot exploit as effectively.

The raw+zstd column confirms that delta encoding is essential: removing the delta stage reduces zstd's compression ratio by 10–44% across modalities. However, even with delta encoding, zstd cannot match MIC's RLE+FSE backend. This validates the design decision to use a domain-specific RLE stage that operates on 16-bit symbols rather than relying on a general-purpose byte-level compressor.

9 MED Predictor Comparison

The JPEG-LS standard [4] uses the Median Edge Detector (MED) predictor, which selects among three candidates based on edge orientation:

$$\hat{x} = \begin{cases} \min(a, b) & \text{if } c \geq \max(a, b) \\ \max(a, b) & \text{if } c \leq \min(a, b) \\ a + b - c & \text{otherwise} \end{cases} \tag{8}$$

where a is the left neighbor, b is the top neighbor, and c is the top-left diagonal neighbor. MED adapts to horizontal, vertical, and diagonal edges, whereas MIC's average predictor $\hat{x} = \lfloor (a + b)/2 \rfloor$ treats all directions equally.

To evaluate whether the more sophisticated MED predictor improves MIC's compression, we implemented a MED-based delta encoder with the same overflow delimiter scheme and ran it through the full RLE+FSE pipeline. Table 16 compares compression ratios.

The MED predictor shows modest compression improvements on some modalities (CT: +3.1%, MG3: +2.9%, MG1/MG2: +1.5%) but slight losses on others (XR: −0.2%, MG4: −1.7%). Across all eight test images, the geometric mean improvement is approximately +0.9%.

However, MED incurs a meaningful decompression speed penalty. The MED predictor requires access to three neighbors (left, top, top-left diagonal) versus two for the average predictor (left, top). The diagonal dependency prevents the branch-free interior loop optimization used in MIC's average-predictor decoder (Section 3), adding one extra memory load and two comparisons per pixel. In our measurements, MED decompression is approximately 1.5–2× slower than the average predictor.

MIC retains the average-of-neighbors predictor as the default because: (1) the compression ratio gains are small and inconsistent across modalities, (2) the decompression speed penalty is significant, and (3) MIC's design philosophy prioritizes

Table 15: Compression ratio: MIC (Delta+RLE+FSE) vs. Delta+zstd at varying levels and raw+zstd (no delta).

Image	MIC	d+zstd -1	d+zstd -3	d+zstd -19	raw zstd-3
MR	**2.35**×	1.75×	1.82×	1.95×	1.65×
CT	**2.24**×	1.71×	1.89×	2.03×	1.79×
CR	**3.63**×	2.70×	2.95×	3.27×	2.05×
XR	**1.74**×	1.43×	1.43×	1.43×	1.32×
MG1	**8.57**×	6.19×	6.37×	7.07×	5.77×
MG2	**8.55**×	6.18×	6.36×	7.04×	5.75×
MG3	**2.29**×	1.71×	1.89×	2.09×	1.50×
MG4	**3.47**×	2.80×	2.87×	2.99×	2.57×

Table 16: Compression ratio: avg-of-neighbors predictor (MIC default) vs. MED predictor, both through the RLE+FSE pipeline.

Image	Avg predictor	MED predictor	Diff
MR	2.348×	2.357×	+0.3%
CT	2.238×	2.306×	+3.1%
CR	3.628×	3.632×	+0.1%
XR	1.738×	1.734×	−0.2%
MG1	8.566×	8.690×	+1.5%
MG2	8.553×	8.678×	+1.5%
MG3	2.289×	2.356×	+2.9%
MG4	3.474×	3.415×	−1.7%

decompression throughput over marginal ratio improvements.

10 Discussion

Design philosophy. MIC deliberately avoids complex prediction models (context modeling, Markov predictors) in favor of a simple three-stage pipeline where each stage has $O(n)$ complexity and minimal branching. This design sacrifices a small amount of compression ratio (compared to JPEG 2000) in exchange for dramatically higher decompression throughput. For clinical workflows where images are compressed once and decompressed many times (archival → viewing), this tradeoff is strongly favorable.

Bit-depth adaptivity. A notable feature of MIC is its automatic adaptation to the effective bit depth of the image. All thresholds and delimiters are derived from the actual maximum pixel value via $\lceil \log_2(\max + 1) \rceil$, eliminating the need for separate 8-bit and 16-bit code paths. This simplifies the implementation and ensures optimal performance across the full range of DICOM bit depths (8–16 bits).

Parallelism strategies. MIC exposes parallelism at three granularities that address different use cases. *Frame-level parallelism* (MIC2 independent mode) decompresses multiple frames concurrently with zero per-frame overhead, reaching 16 GB/s at 64 cores. *Tile-level parallelism* (MIC3) uses a goroutine pool to encode/decode WSI tiles concurrently. *Strip-level parallelism* (PICS) partitions a single image into horizontal strips for multi-core decompression of a single greyscale image. Each strategy trades a small amount of compression ratio for throughput; PICS offers 2.5–3.9× speedup at a ratio cost of <4% on small images and zero cost (or slight improvement) on large images.

Limitations. MIC does not support lossy compression, progressive decoding, or region-of-interest coding for greyscale images. These capabilities are available in JPEG 2000 and may be important for bandwidth-limited telemedicine applications. The wavelet transform, already implemented and benchmarked, provides a natural foundation for a future lossy mode.

For WSI images, MIC3's tiled architecture pro-

vides tile-level random access and pyramid-level multi-resolution support, addressing the region-of-interest and progressive delivery requirements specific to digital pathology.

Comparison with HTJ2K and JPEG-LS. Section 6.7 presents a fair in-process comparison against HTJ2K (OpenJPH [12]), correcting earlier subprocess-based measurements that inflated apparent HTJ2K latency. When measured fairly, HTJ2K decompresses ∼1.8× faster than MIC-Go, but MIC's C four-state implementation closes to within 0.87× (geometric mean), and MIC's Wavelet V2 SIMD pipeline exceeds HTJ2K on all eight modalities. Section 6.8 compares against JPEG-LS (CharLS [13]), showing that JPEG-LS achieves 1–16% better compression ratios but MIC 2-state decompresses 1.4–2.6× faster and MIC 4-state decompresses 1.5–3.5× faster on 6/8 modalities (4-state beats JPEG-LS on all 8). At 64 cores, MIC's per-frame parallelism scales to 16 GB/s (best case), an advantage enabled by the pipeline's simplicity and independence across frames.

11 Conclusion

We have presented MIC, a lossless medical image codec that achieves compression ratios of 1.7×–8.6× on greyscale DICOM images and 3–5× on RGB pathology tissue tiles, with decompression throughput up to 16 GB/s on mammography (best case; geometric mean ≈7.5 GB/s across modalities on 64 cores). The codec's three-stage pipeline (delta prediction, run-length encoding, FSE entropy coding) is designed for simplicity and speed: each stage processes the data in a single pass with minimal branching, enabling decompression performance that scales with memory bandwidth.

A four-state FSE decoder that runs four independent ANS state machines with platform-specific assembly kernels (BMI2 on amd64, scalar on arm64) delivers 66–142% faster isolated FSE decompression and 20–75% faster end-to-end pipeline throughput by exploiting instruction-level parallelism. The two-state variant is pure Go and benefits all platforms without assembly.

MIC supports four formats for different clinical use cases: single-frame (MIC1), multi-frame with temporal prediction (MIC2), tiled whole slide imaging with pyramid levels (MIC3), and PICS (Parallel Image Compressed Strips) for multi-core decompression of a single image at 2.5–3.9× single-thread throughput on images ≥7 MB. The implementation includes browser-based JavaScript and WebAssembly decoders for web viewing applications.

Our evaluation includes fair in-process comparisons against three established codecs:

- **HTJ2K** (OpenJPH): HTJ2K decompresses 1.8× faster than MIC-Go single-threaded, but MIC's C four-state implementation closes to within 0.87×, and MIC's Wavelet V2 SIMD pipeline exceeds HTJ2K in decompression speed on all 8 modalities. MIC achieves comparable or better compression ratios.
- **JPEG-LS** (CharLS): JPEG-LS achieves 1–16% better ratios due to adaptive prediction, but MIC 2-state decompresses 1.4–2.6× faster on 6/8 modalities; MIC 4-state decompresses 1.5–3.5× faster and beats JPEG-LS on all 8 modalities.
- **Delta+Zstandard**: MIC's 16-bit-native RLE+FSE outperforms byte-oriented Delta+zstd by 10–22% on all modalities, even at zstd's highest compression level.

A five-level 5/3 integer wavelet alternative with subband-order scanning matches or exceeds both Delta+RLE+FSE and HTJ2K compression ratios on 7/8 modalities, with SIMD-accelerated decompression exceeding HTJ2K speed on all 8 modalities. This wavelet pipeline provides a natural foundation for future lossy and progressive modes.

The PICS format demonstrates that strip partitioning provides orthogonal and additive throughput gains: PICS-8 reaches 2.4 GB/s on MG images (12-core ARM64 workstation) while maintaining compression ratios comparable to or better than single-strip MIC on large images.

These results suggest that for lossless medical image compression, a domain-specific 16-bit entropy coding pipeline—whether paired with simple delta prediction for maximum speed or a

wavelet transform for maximum ratio—is more effective than both byte-oriented general-purpose compressors and the context-heavy approaches of established standards.

MIC is open-source and available at https://github.com/pappuks/medical-image-codec.

Availability. Source code, test data, and benchmark scripts are available under an open-source license at the URL above.

References

[1] NEMA, "Digital Imaging and Communications in Medicine (DICOM)," https://www.dicomstandard.org/, 2024.

[2] D. S. Taubman and M. W. Marcellin, "JPEG2000: Image Compression Fundamentals, Standards and Practice," Springer, 2002.

[3] D. S. Taubman, "High throughput JPEG 2000 (HTJ2K): Algorithm, performance evaluation, and potential," *Proc. SPIE*, vol. 11137, 2019.

[4] M. J. Weinberger, G. Seroussi, and G. Sapiro, "The LOCO-I lossless image compression algorithm: Principles and standardization into JPEG-LS," *IEEE Trans. Image Process.*, vol. 9, no. 8, pp. 1309–1324, 2000.

[5] S. Mahapatra and K. Singh, "An FPGA-based implementation of multi-alphabet arithmetic coding," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 54, no. 8, pp. 1678–1686, Aug. 2007.

[6] J. Duda, "Asymmetric numeral systems," *arXiv preprint arXiv:0902.0271*, 2009.

[7] Y. Collet, "Finite State Entropy—A new breed of entropy coder," https://github.com/Cyan4973/FiniteStateEntropy, 2013.

[8] Y. Collet and M. Kucherawy, "Zstandard Compression and the 'application/zstd' Media Type," RFC 8878, Internet Engineering Task Force, 2021.

[9] E. S. Schwartz and B. Kallick, "Generating a canonical prefix encoding," *Commun. ACM*, vol. 7, no. 3, pp. 166–169, 1964.

[10] H. S. Malvar and G. J. Sullivan, "YCoCg-R: A color space with RGB reversibility and low dynamic range," *Joint Video Team (JVT) Doc. JVT-I014*, 2003.

[11] D. Le Gall and A. Tabatabai, "Sub-band coding of digital images using symmetric short kernel filters and arithmetic coding techniques," *Proc. IEEE ICASSP*, pp. 761–764, 1988.

[12] A. N. Aous, "OpenJPH: An open-source implementation of High-Throughput JPEG 2000," https://github.com/aous72/OpenJPH, 2024.

[13] Team CharLS, "CharLS: A C/C++ JPEG-LS library implementation," https://github.com/team-charls/charls, 2024.